VIETNAMESE – GERMAN UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

Frankfurt University of Applied Sciences

Faculty 2: Computer Science and Engineering

Real-time fish 3d stereo detection and length estimation on OAK-D stereo camera

Full name: Tra Dang Khoa

Matriculation number: 17637

First supervisor: Dr. Vo Bich Hien

Second supervisor: Dr. Nguyen Tuan Cuong

BACHELOR THESIS

Submitted in partial fulfillment of the requirements for the degree of Bachelor Engineering in study program Computer Science, Vietnamese - German University, 2025

Binh Duong, Viet Nam

Disclaimer and Acknowledgement

This thesis is done under my own decisions and research so any conclusion, ideas and suggestions will be fall on my responsibility. Vietnamese-German University is not responsible for any the content in this thesis. This thesis is also a collaboration with other VGU student, Mr Nguyen Huu Minh Quang (ID: 1752) intake 2020.

To commit to the academic transparency and integrity, in the thesis, I admit that I have used the 2 gen AI: Gemini, Deepseek for searching for science paper, suggestions and grammar fix. As well as code debug and general question about the project. But the rest of the thesis and the code are written and tested by my own.

I thank Mr. Nguyen Huu Minh Quang – VGU student (ID: 17452) - for providing the trained yolov8n pose estimation and database setup. I would like to Thanks Dr. Vo Bich Hien for providing OAK-D camera which enable me to do this thesis and Dr.Nguyen Tuan Cuong for ideas and suggestions.

Table of Contents

1. Abstract:

This thesis presents the attempt at designing and implementing a real-time detection system for automated fish behavior analysis and biometric measurement. The system utilized a stereo vision camera (OAK-D) to perform simultaneous 2D detection, pose estimation, and 3D spatial calculation. A YOLOv8 nano pose estimation model, converted for the target hardware, identifies fish and their key anatomical points (head and tail). These 2D detections are fused with real-time depth data to calculate accurate 3D positions and body length. A multi-object tracker (ByteTrack) maintains individual fish ID across frames, enabling the measurement of movement metrics such as distance traveled and then used for analyzing activity status. All processed data—including size, 3D location, and behavior state are logged to a cloud database for storage and future analysis. The entire pipeline, was created runs efficiently as much as possible on the edge device, demonstrating a potential low-power solution for aquaculture monitoring and ethological research. This project was tested on low-end laptop with windows operating system as host for the camera and the tested environment was outside controlled fish tank.

The thesis is under collaboration with another student – Mr Nguyen Huu Minh Quang where his main task is to provide the trained model, setup database and provide tracking, status checking and distance traveled code for reference.

2. Introduction

Automated monitoring in aquaculture and aquatic research requires systems that can operate continuously with minimal human intervention, providing quantifiable data on animal health, size, and behavior. Traditional methods of data collection in this field are often manual, subjective, and labor-intensive, requiring physical handling that can be invasive and stressful to the marine life. Alternatively, automated solutions exist but frequently rely on expensive, heavy, and complex equipment that is not feasible for widespread deployment [1]. This project develops an integrated hardware-software system in the attempt to address this gap.

The main idea is to create edge-based system that uses computer vision and stereo depth sensing to detect fish, estimate their length in three-dimensional space, track their individual movements over time, and classify their activity state based on movement patterns. Unlike previous iterations or simpler approaches that rely solely on bounding box detection—which often fails to account for fish body curvature and rotation—this system integrates YOLOv8 pose estimation. The system is built around the DepthAI platform, which allows for the encapsulation of the entire vision pipeline—from image capture and neural network inference to depth calculation—within a single embedded device, reducing latency and power consumption on the Host which in this project is a cheap and low-end laptop.

3. Background and related work

This Thesis is an evolution of a work of mine from the internship for Dr.Vo Bich Hien. The task of my internship was implementing the stereo depth detection feature of the OAK-D camera for fish detection. My old work was referenced from the "Automatic length estimation of free-swimming fish using an underwater 3D range-gated" developed by Petter Risholm, Ahmed Mohammed, Trine Kirkhus, Sigmund Clausen, Leonid Vasilyev, Ole Folkedal, Øistein Johnsen, Karl Henrik Haugholt, Jens Thielemann [2]. The reasearch paper was about using stereo vison for fish length measurement without having to manually measure each fish and being invasive to the fish's environment. Thus, creating a free environment for the fish to grow well while we automatically collect the data of each fish. While the conceptual goal aligned with Risholm et al., the practical implementation differed significantly due to resource constraints technical feasibility and own limited capability at the time. Instead of using masking model, I used bounding box detection and follow pinhole formula [3] to calculate real length of fish. The implementation posed many limitations. not accounted for fish rotation and the accuracy was low.

After finishing the internship, I stumbled upon a research paper called "Key point detection method for fish size measurement based on deep learning" by Yaozhen Yu, Hao Zhang, Fei Yuan. This research is about using pose estimation machine learning to create two keypoints (head, tail) on the fish, then get 3D coordinate of those keypoints and using curve fitting algorithm to calculate the length of each detected fish [1]. The result in the research show to have increased in accuracy and the system robustness. This inspire me to rework my old stereo detection system and create this entire thesis. Combining with idea of edge computing, I decided to use yolov8 nano

4. The hardware and camera setup

The system is built upon the edged-based camera module called OAK-D developed by Luxonis company. This device houses 2 mono cameras for stereo vison, one RGB camera and an onboard Intel Myriad X vision processing unit (VPU). The stereo baseline enables the calculation of depth maps via triangulation. The RGB stream is used for object detection, while the synchronized stereo pair and depth map are used for projecting 2D detections into 3D space. All neural network inference and depth processing are offloaded to the VPU, freeing the host computer from intensive image processing tasks and enabling efficient real-time performance. [4] [5]

Key feature and performance:

- 4 TOPS of processing power (1.4 TOPS for AI - RVC2 NN Performance)
- Run most AI model, even custom architected/built ones (models need to be converted)
- Computer vision: warp (undistortion), resize, crop via ImageManip node, edge detection, feature tracking.
- Stereo depth perception with filtering, post-processing, RGB-depth alignment, and high configurability
- Object tracking: 2D and 3D tracking with ObjectTracker node but for dectection only
- Base line: 75 mm
- Ideal depth range: 70cm - 12m.
- Power consumption: Up to 5W.
- below 4m: below 2% absolute depth error
- 4m - 7m: below 4% absolute depth error
- 7m - 10m: below 6% absolute depth error [4] [5]

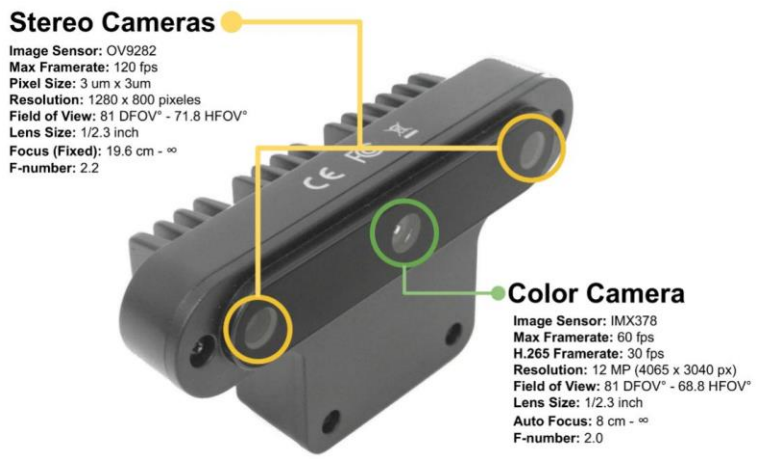


Figure 1. OAK-D stereo camera [4]

4.1. Stereo limitation:

Beyond its limited effective range, stereo vision consistently struggles with thin or fine objects, as the example below illustrates

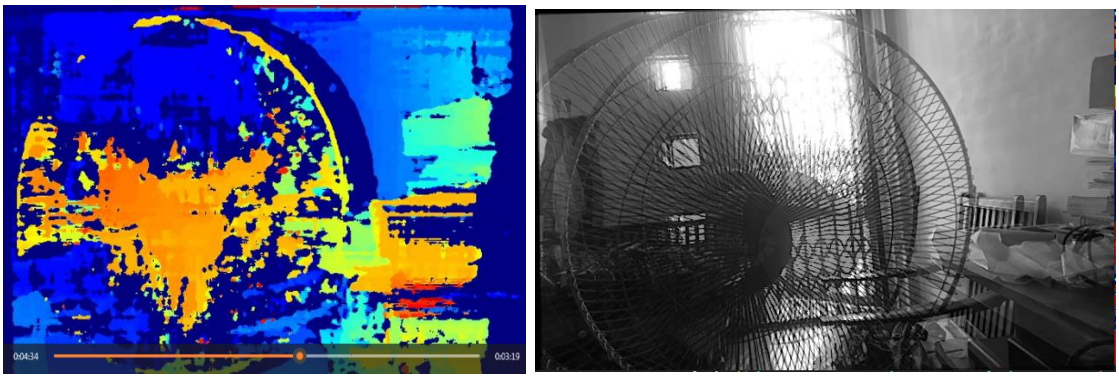


Figure 2. struggle detect each individual line of the cage of a fan

This isn't merely a theoretical constraint—it directly impacted our ability to resolve individual elements in complex scenes. Furthermore, the system's depth accuracy seems heavily influenced by lighting and the texture of objects themselves. Well-defined features are needed for the algorithm to find correspondences between the two camera views. A smooth, featureless surface like a blank wall often results in poor or noisy depth data, a limitation we frequently encountered during testing. In our context, this implies that a fish's visibility—its contrast against the background and the texture of its scales—may be just as important as its physical presence for reliable measurement.

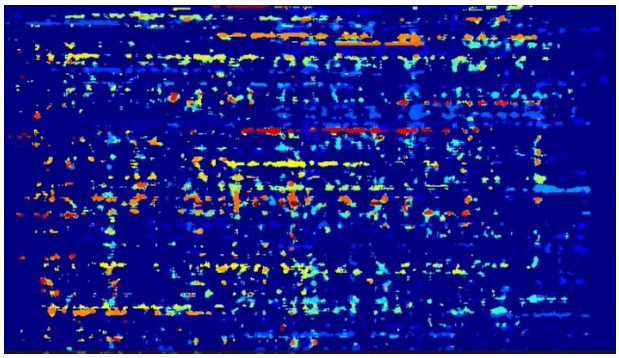


Figure 3. In the dark

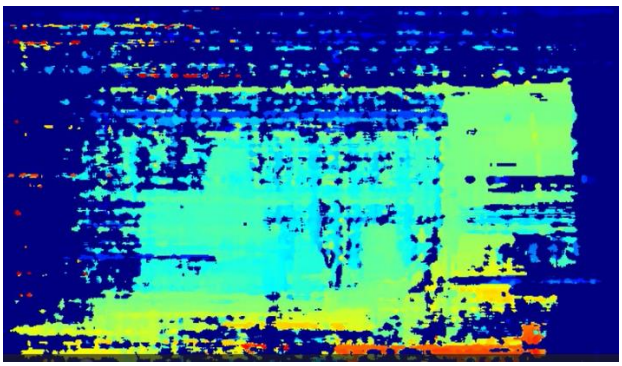


Figure 4. low light condition

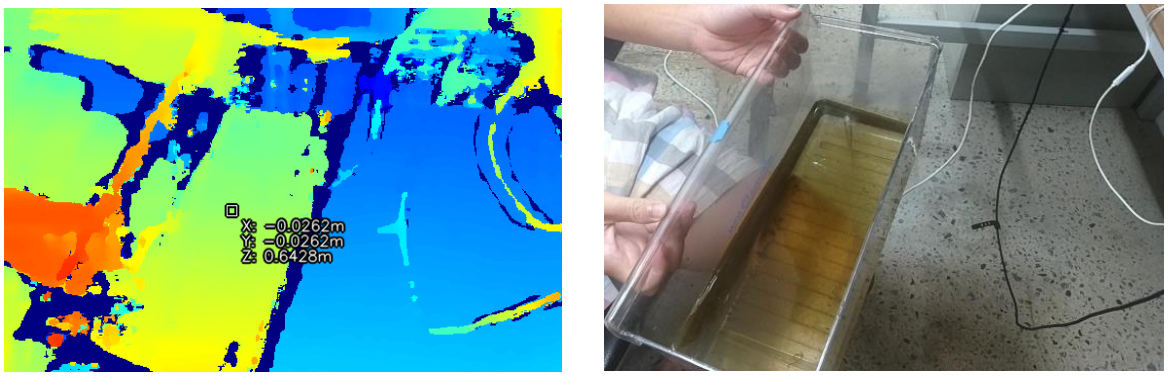


Figure 5. Muddy water

Figure 6. Clear water

A clear difference in performance is evident between Figures 5 and 6. In the turbid, muddy water, the stereo depth map essentially fails to resolve the fish's form, resulting in a sparse or absent point cloud for the animal. By contrast, in clear water, the system generates a coherent depth profile along the fish's body. This stark contrast underscores a fundamental dependency: stereo matching algorithms require textured surfaces and visible features to establish correspondence between the two camera views. Murky water, which obscures these features, severely degrades that process. Consequently, one should anticipate significant performance variation in real-world settings, where water clarity is rarely constant or optimal

4.2.The basic of stereo vison:

The stereo vision works by calculate the disparity (shift) of objects in two images, one come from mono camera left and other from mono camera right. Mimic the functionality of real human eyes. For more technical view, disparity is the pixel distance between the same point in the left and right images of the stereo pair camera. The OAK-D camera calculates disparity for every pixel in the mono frame, assigning a disparity value with a certain confidence level. All of this process occurs inside the StereoDepth node.[4] [5]

In the StereoDepth node, the formula below is used to calculate the depth using disparity

depth = focal_length_in_pixels * baseline / disparity_in_pixels [5]

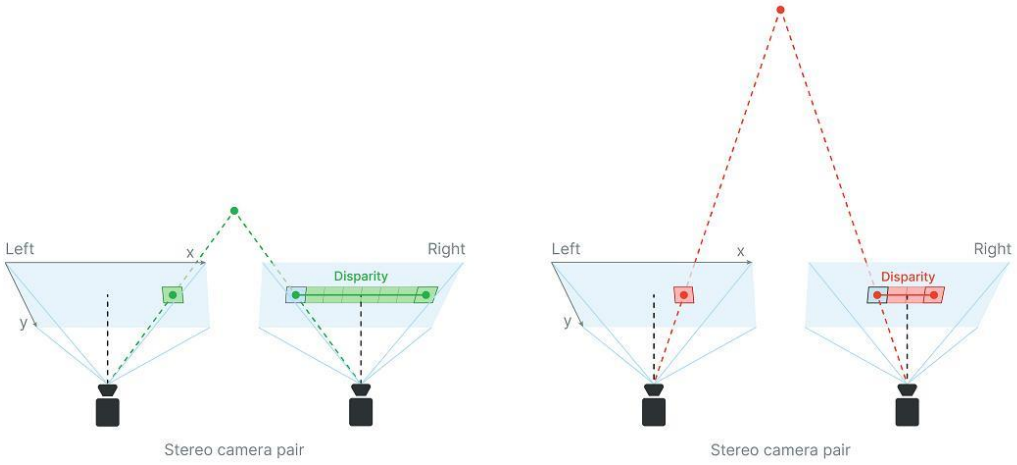


Figure 7. stereo depth perception [4]

Even though the camera has VPU to process AI and depth calculation it still needs a host to host the source code.

The host used for this thesis was hp laptop with dual core i5 7th generation CPU with dual ram 4gb ddr3. While this wouldn't be considered as a true edge device such as Raspberry pi 3 or more, it was employed to fulfill the functional role of an Edge Node within this work's architecture [6], because of the lack of dedicated edge devices.

Here is the overview hardware pipeline:

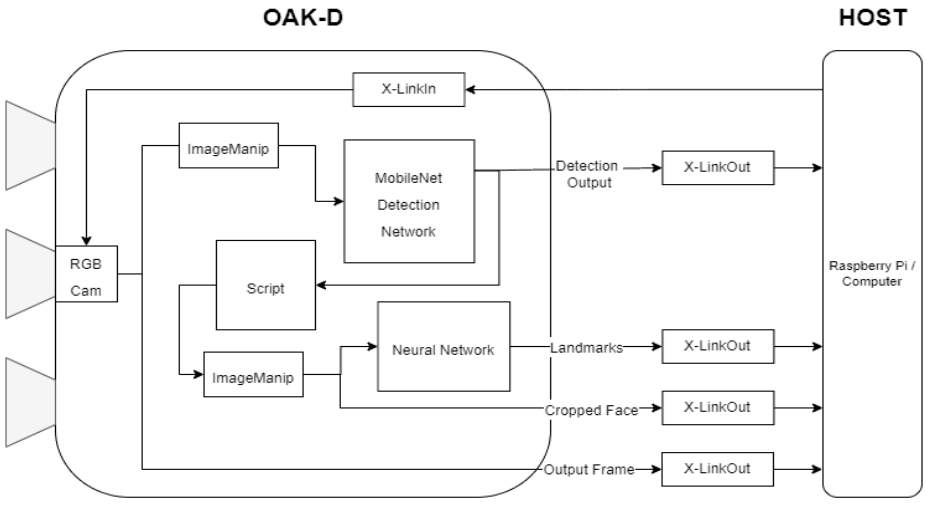


Figure 8. OAK-D general pipeline [5]

4.3. Calibration

4.3.1. Calibration overview

For Stereo depth detection system to work and function accurately, OAK-D camera required to be precisely calibrated. This process mathematically models the camera's geometry to correct optical distortions and align the two stereo cameras. Obtaining intrinsic and extrinsic parameters of cameras as well as their distortion coefficients for depth calculation. The focal length can be obtained directly from the calibration data for depth calculation from basic of stereo vision section.

Many OAK-D camera come with its own factory calibration data stored in their internal EEPROM. Thus, calibration often not needed. Only when, the effectiveness of the calibration data start to deteriorate or environment changes or Physical Deformation of the camera lens. [5]

4.3.2. Intrinsic Parameters (The Lens & Sensor)

These define how a 3D point in the world is mapped to a 2D pixel on the image sensor:

- Focal length (f_x,f_y): Determines the scale of the image.
- Optical center(c_x,c_y): The point where the optical axis intersects the image plane.
- Distortion Coefficients: Lenses (especially wide-angle ones) curve light, causing "barrel" or "pincushion" distortion. Calibration calculates these coefficients (k_1, k2, p_1, p_2, etc.). [5]

4.3.3. Extrinsic Parameters (The Position)

These define the relationship between the Left and Right cameras:

- Rotation (R): How the right camera is rotated relative to the left.
- Translation (T): The precise distance (baseline) between the two camera centers. [5]

4.3.4. Calibration Workflow using Luxonis Utilities

To perform a custom calibration, you utilize the depthai library and the Charuco board (a hybrid of a Chessboard and ArUco markers). The Charuco board is superior to a standard chessboard because it works even if part of the board is occluded, allowing for more robust detection at the edges of the frame.

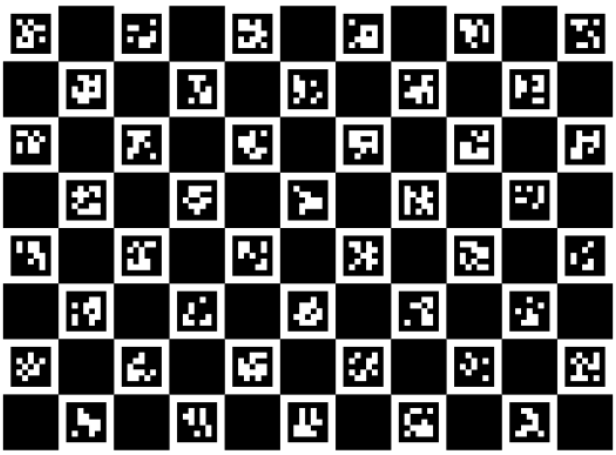


Figure 9. 8x11 330x230 DICT_5x5 Charuco board

Clone the Luxonis' github that contain calibration.py. Run the calibration script then follow guide of the application to do calibration. Prepare the Charuco board target by Print the Charuco Board. Ensure measure the square size of the printed board physically (in millimeters) to ensure scale accuracy. After the process is done, the calibration data will be store inside internal EEPROM of the camera.

5. System architecture and implementation

5.1. Overall System Architecture

The system is constructed around a modular, sequential pipeline that transforms raw stereo imagery into structured, logged observations. This design, which might be described as a directed acyclic graph of processing stages, attempts to balance computational efficiency on the edge device with the flexibility needed for real-time analysis. While not without its trade-offs—particularly regarding latency between modules—the architecture seeks to isolate distinct responsibilities: data acquisition, inference, spatial calculation, tracking, measurement, and persistence. The flow of data through these modules is largely unidirectional, a design choice that simplifies the streaming model but may also introduce bottlenecks if any single stage becomes overloaded.

The process begins with the OAK-D hardware pipeline, configured within MainTrackCurrent.py. Here, synchronized image streams are captured: the RGB feed for object detection and the rectified stereo pair for depth computation. The neural network inference, powered by a converted YOLOv8n-pose model whose deployment is managed by model_utils.py and ModelConvert.py, executes directly on the camera's VPU. This stage outputs 2D bounding boxes and the pixel coordinates for two keypoints per detection—head and tail. Notably, this sparse keypoint selection represents a deliberate compromise to reduce

computational load, though it occasionally struggles with ambiguous poses where head-tail orientation is unclear.

Subsequently, these 2D detections are passed to the tracking.py module, which houses the ByteTrack algorithm. This component is responsible for maintaining individual fish identities across frames. It converts the proprietary DepthAI detection format into a standardized structure, associates tracks, and crucially, re-attaches the keypoint data that the tracker would otherwise strip away. The persistence of identity is foundational for all subsequent behavioral metrics.

Following tracking, the system enters a spatial calculation phase. The 2D keypoints from each tracked detection are projected into 3D space. This is achieved by defining Regions of Interest (ROIs) around each keypoint coordinate and querying the synchronized depth map via a SpatialLocationCalculator node. The resulting 3D point cloud for each fish—comprising head, tail, and interpolated mid-body points generated by fish_size_estimate.py—is then passed to the curve-fitting algorithm. This algorithm, implementing the polynomial surface method described in the related work, attempts to approximate the animal's body curvature, providing a length estimate that is more faithful than a simple Euclidean distance between endpoints. One might observe that the accuracy of this step is inherently contingent on the quality of the underlying depth data, which can degrade in turbid water or under low texture.

Concurrently, the TrackerHandler calculates a 3D centroid from the head and tail points. This centroid, alongside a timestamp, feeds a simple time-based motion heuristic to determine activity status (active, lethargic, dead). The cumulative distance traveled is updated by measuring the displacement of this centroid between successive appearances of the same track ID.

All derived metrics—fish ID, length, 3D location, distance traveled, and status—are then passed to the DataBaseHandler.py module. This component handles the conversion of NumPy types to native Python formats and manages the upsert operation to a cloud MongoDB database, ensuring each fish's record is continuously updated. A separate, non-blocking video recording stream, managed by record.py, can be toggled at runtime, storing the annotated visual feed for later review without impeding the main processing loop.

Orchestrating this entire workflow, MainTrackCurrent.py also instantiates a SystemMonitor (from Monitor.py). This monitor runs in a separate thread, sampling host CPU usage and calculating processing FPS, which provides a pragmatic, if limited, view of system performance during development and testing.

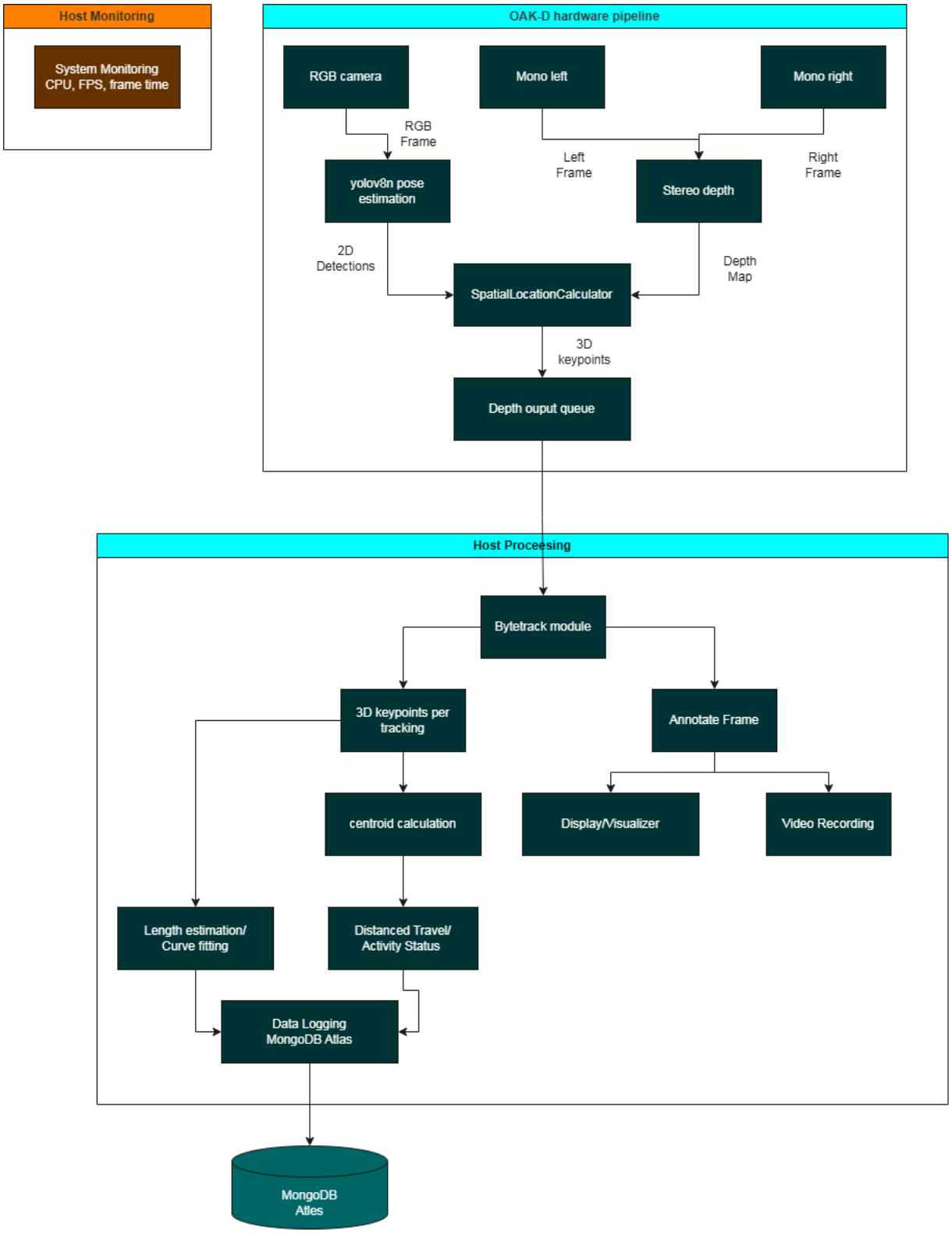


Figure 10. System Architecture

5.2. Machine learning model

For fish detection and anatomical landmark localization, the system implements a YOLOv8n-pose model (YOLO – You Only Look Once). This lightweight pose-estimation variant outputs pixel coordinates for predefined keypoints—here, the head and tail of each fish—rather than simple bounding boxes. Adapted from the broader YOLOv8 architecture, the nano version is intentionally pared down to reduce parameter count and computational demand, a practical necessity for deployment on edge devices like the OAK-D's Myriad X VPU [reference to YOLOv8 architecture or DepthAI deployment papers]. While the thesis does not focus extensively on model architecture, it is worth noting that the network maintains the familiar backbone-neck-head design, with the pose head generating heatmaps for each keypoint through a series of convolutions and upsampling layers. This design allows the model to retain reasonable accuracy while meeting the strict latency and power constraints of real-time, on-device inference.

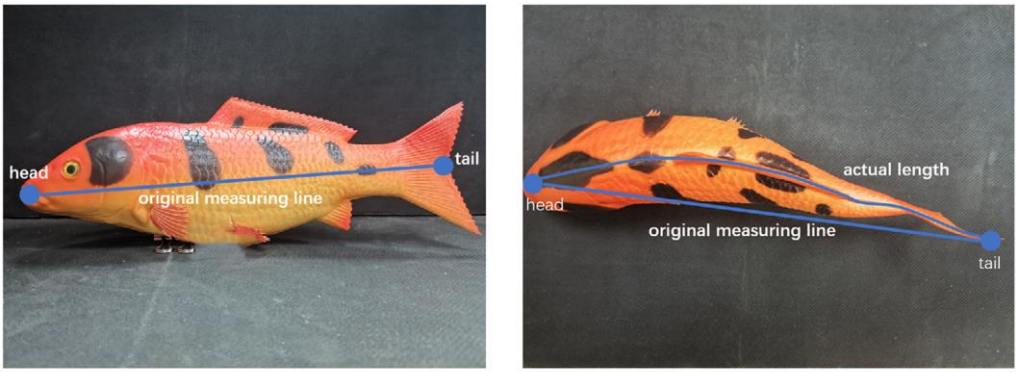
In operational terms, the model delivers a usable trade-off between speed and precision. Limiting the keypoints to only two per fish reduces computational overhead, which helps sustain frame rates during continuous monitoring. Nevertheless, the approach exhibits certain vulnerabilities. Under partial occlusion or during rapid lateral turns—common in active shoaling—keypoint confidence can drop, and occasional mislabeling of head and tail points has been observed, particularly when fish are seen at oblique angles.

Figure 10. one of the problems of key point detection

The specific instance of the model deployed in this work was trained by Mr. Nguyen Huu Minh Quang, who prepared the dataset and optimized the network for fish keypoint detection. My contribution involved converting the trained PyTorch model into a format compatible with the DepthAI pipeline—a necessary step to execute inference directly on the OAK-D's VPU. This conversion, while straightforward using Luxonis's tools, required careful attention to input dimensions and layer compatibility to ensure the model ran correctly on the embedded hardware.

5.3. Curve fitting algorithm

A fundamental challenge in automated fish sizing is that the animal's body is rarely perfectly straight. A simple Euclidean distance between the 3D head and tail points—while computationally trivial—will systematically underestimate the true length whenever the fish exhibits any curvature, a common occurrence during swimming or turning. The system, therefore, incorporates a curve-fitting stage to approximate the body's arc and compute a more biologically accurate length measurement. This approach is inspired by the method detailed in the related work [1]., which moves beyond linear measurement to fit a polynomial surface to the fish's spine.



(a) Original measuring length (b) Actual length

Figure 11. measure length and actual length [1]

The core concept is to treat the set of retrieved 3D points—originating from the head, tail, and interpolated mid-body points—not as a disconnected cloud but as samples along a smooth, continuous curve in space. The algorithm implemented in `fish_size_estimate.py` primarily uses a polynomial fitting method. It posits that the depth (z-coordinate) of a point along the fish's body can be modeled as a function of its x and y planar coordinates, expressed as z = F(x, y). For a second-degree polynomial, this function takes the form:

$$F(x, y) = a_0 + a_1x + a_2y + a_3x2 + a_4xy + a_5y^2$$

The coefficients (a_0, a_1, ..., a_5) are solved via a least-squares regression, minimizing the total squared error between the actual measured z-values of our 3D points and the values predicted by the polynomial surface. This process effectively finds the smooth surface that best explains the observed depth variations along the fish's body line.

However, fitting the surface is only the first step. The actual length we seek is not a straight line in 3D space but the arc length of the curve that lies on this fitted surface and passes through our points. To obtain this, the algorithm parameterizes the curve. It typically uses the x-coordinate as the parameter (s), assuming the fish's body primarily extends along this axis.

The y-coordinate is then expressed as a linear function of x (based on the head-tail line), and the z-coordinate is given by our polynomial F(x, y). The derivatives $\frac{dx}{ds}$, $\frac{dy}{ds}$ and $\frac{dz}{ds}$ are computed either analytically from the polynomial or via finite differences. Finally, the arc length L is calculated by numerically integrating the speed along this parameterized path:

$$L = \int_{S_{start}}^{S_{end}} \sqrt{(\frac{dx}{ds})^2 + (\frac{dy}{ds})^2 + (\frac{dz}{ds})^2}\, ds$$

In practice, this integration is performed using SciPy's quad function. The result is a length estimate that accounts for the body's bend in three dimensions, which should, theoretically, be closer to the manual measurement one would obtain with a flexible tape along the fish's contour.

Implementing this seemed straightforward in theory, but the reality of real-time, noisy data demanded concessions. The polynomial fit, for instance, can become unstable or produce nonsensical curves if the depth data for a keypoint is erroneous—say, if the stereo matching fails on a low-texture patch of the fish's side and returns a `z` value of zero or near-zero. The code therefore includes a validation gate, discarding points with unreliable depth before fitting. Furthermore, the numerical integration adds a non-trivial computational overhead. While not severe for a few fish, it's a consideration for scaling. This led to the inclusion of a simpler fallback: the `simplified_curve_length` function. This method simply connects the sequential 3D points with straight line segments and sums their Euclidean distances—a polyline approximation. It's less sophisticated than the polynomial arc integral and tends to slightly underestimate length on smooth curves, but it's far more resilient to noise and runs almost instantly. The system logic often defaults to this polyline method when too few valid points are available, a pragmatic choice that prioritizes a stable, reasonable estimate.

5.4. Fish status tracking

Beyond simply detecting and measuring fish, the system attempts to infer their behavioral state—classifying each individual as active, lethargic, or dead based on its movement pattern. This status logic provides a simplified, real-time indicator of potential welfare issues or environmental stress, shifting the system from pure metric collection toward basic behavioral analysis.

The implementation draws from the earlier work of Mr. Nguyen Huu Minh Quang, but modifies a core assumption. In the reference implementation (track_video2 .py) [7], behavioral status is determined using a frame-based threshold. A fish is considered lethargic or dead after remaining motionless for a predefined number of frames (e.g., 5 minutes converted to frames using a fixed FPS). While straightforward, this method implicitly ties

behavioral time scales to the video's frame rate. A system experiencing slowdowns or processing variable frame rates would inadvertently alter the meaning of "5 minutes of inactivity"—a fish might be flagged as dead sooner simply because the host CPU was overloaded, not because of its actual biology.

To decouple behavioral judgment from processing performance, the current system employs a time-based threshold. Here, status transitions are triggered by actual elapsed seconds, monitored via time.time(). A fish's last_active_time is updated whenever its 3D centroid movement exceeds a minimal motion threshold (e.g., 0.05 meters). If the time since this last activity surpasses, say, 30 seconds, the fish is classified as lethargic; if it exceeds 60 seconds, it is marked dead. This approach more closely aligns with real-world ethological observation, where a minute of stillness means the same regardless of how many frames were captured. One practical drawback, however, is its dependence on accurate system clocks and synchronized timestamps across the pipeline—a generally safe assumption on a single host, but a point of failure in distributed setups.

Both methods share a common vulnerability: they rely on a single, simplistic heuristic. The classification is based solely on displacement of a computed centroid, ignoring other potential indicators of state like body orientation, fin movement, or respiratory cues. Consequently, a fish holding perfectly still in a current might incorrectly be flagged as lethargic, while minor vibrations or tracking jitter could reset the activity timer for a genuinely stressed animal. In practice, this means the status output should be interpreted as a coarse filter—useful for triggering human review rather than making autonomous decisions.

5.5. Detail code breakdown

5.5.1. Monitor.py – Host System Performance Monitoring

This module provides real-time performance monitoring of the host system, necessary for testing and performance checking of the host.

Key Functions:

- __init__(self, history_size=30): Initializes monitoring with configurable history buffers for frame times, CPU usage, and FPS. Starts a daemon thread (_monitor_cpu) that continuously samples CPU usage.
- _monitor_cpu(self): Background thread function that uses psutil.cpu_percent() to sample CPU usage at 500ms intervals, appending to a circular buffer.
- start_frame(self): Called at the beginning of each processing frame. Records start time and increments frame counters.
- end_frame(self): Called at frame completion. Calculates frame processing time, updates FPS calculation (if ≥1 second elapsed), and returns frame time.

- get_stats(self): Computes and returns comprehensive statistics including current FPS, frame count, average/min/max frame times, current/avg CPU usage
- stop(self): Gracefully stops monitoring thread and cleans up resources

Purpose: Provides performance telemetry to identify bottlenecks and validate system stability of the host.

5.5.2. model_utils.py - Model Management and Conversion

Handles the task of model deployment, including automatic discovery and conversion of PyTorch models to DepthAI-compatible format.

Key Functions:

- ensure_nn_archive(nn_archive_path, base_dir=None): Core function that:
  - Searches for existing RVC2/TAR.XZ archives using glob patterns
  - If none found, automatically prompts user for PyTorch (.pt) model selection
  - Supports multiple selection methods: direct path, directory browsing, pattern matching
  - Automatically selects single .pt file if only one exists in workspace
  - Calls ModelConvert.convert_model() to convert selected model
  - Attempts to extract archive path from conversion return value
  - Falls back to searching workspace for generated archives
  - Returns resolved archive path or raises FileNotFoundError if not found anything

Purpose: Abstracts away model conversion annoyance such as having to manually change the code when change to the new model, providing a seamless workflow from training (PyTorch) to deployment (DepthAI).

5.5.3. ModelConvert.py - Model Format Conversion

Interface to external model conversion service for transforming YOLO models to RVC2 format.

Key Functions:

- convert_model(model_path, yolo_version):
- Authenticates with conversion API using hardcoded key
- configures conversion parameters (shaves=4, input shape="640 320")
- Specifies model metadata (name, description, class names, tasks)
- Returns converted model object for deployment

Purpose: simple conversion that link to the online converter of Luxonis – OAK-D manufacturer so we don't have to go the website whether there is a new model need to be converted.

5.5.4. tracking.py - Multi-Object Tracking (ByteTrack intergration)

Implements custom object tracking and find 3d centroid point of two 3D keypoints for maintaining fish identities and computing traveled distance.

Key Functions:

- __init__(self, track_thresh=0.3, track_buffer=50): Initializes ByteTrack tracker with configurable parameters (activation threshold, buffer size).
- _nn_results_to_detections(raw_detections, frame_shape): Converts DepthAI's proprietary detection format to standard supervision.Detections format. Processes:
  - Bounding boxes (from rotated_rect.center/size)
  - Confidence scores and class IDs
  - Keypoints (extracts x,y coordinates from nested structures)
  - Scales normalized coordinates to pixel values
  - get_tracked_results(raw_detections, frame_shape): Main tracking pipeline:
  - Converts raw detections to standard format
  - Updates ByteTrack with current detections
  - Re-attaches keypoints to tracked detections (ByteTrack strips custom fields)
  - Returns tracked detections with persistent IDs
- calculate_3d_centroid(head_3d, tail_3d): Computes midpoint between head and tail in 3D space (used for position tracking).

Purpose: Provides consistent object identity across frames and prepares spatial data for downstream analysis.

5.5.5. DataBaseHandler.py - Data Persistence

Manages all database operations for storing fish tracking data to MongoDB Atlas.

Key Functions:

- __init__(self): Sets up connection string and initializes connection.
- _convert_to_mongodb_types(data): Recursive function that converts NumPy types and arrays to MongoDB-compatible Python types.
- connect(self): Establishes connection to MongoDB Atlas, performs connection test, and sets up collection reference.
- save_data_to_db(fish_id, size, distance_traveled_m, is_active, location_m, status):
- Converts all parameters to MongoDB-compatible types

- Constructs document with timestamp and all metrics
- Uses update_one with upsert=True to create/update records
- get_last_known_distance(fish_id): Retrieves previously stored distance for a fish ID (used for cumulative distance calculation).
- close(self): Closes database connection gracefully.

Purpose: Provides persistent storage for long-term analysis and data retrieval.

5.5.6. fish_size_estimate.py - Biometric Measurement

Implements advanced geometric algorithms for accurate fish length estimation from sparse 3D points.

Key Functions:

- generate_intermediate_points(head_2d, tail_2d, num_points=10): Uses linear interpolation to generate evenly spaced points along the line connecting head and tail in 2D space.
- fit_polynomial_curve_and_calculate_length(spatial_points_3d, degree=2): Implements the polynomial surface fitting approach:
    - Fits a polynomial z = F(x,y) to 3D points using least squares
    - Creates parameterized curve representation
    - Computes arc length via numerical integration of $\sqrt{(dx/ds)^2 + (dy/ds)^2 + (dz/ds)^2}$
- simplified_curve_length(spatial_points_3d): Fallback method that sums Euclidean distances between consecutive points (polyline approximation).
- length_estimate(keypoint_1, keypoint_2): Basic Euclidean distance calculation between two 3D points.

Purpose: Provides accurate length estimation even for curved fish postures, moving beyond simple straight-line approximations. Other function are for fallback and testing.

5.5.7. record.py - Video Recording

Threaded video recording system that captures annotated frames without blocking main processing.

Key Functions:

- __init_(self, max_queue_size=30): Sets up frame queue and threading infrastructure.
- start_recording(frame_width, frame_height, output_path, fps):
- Creates output directory

- Initializes OpenCV VideoWriter with MP4V codec
- Starts worker thread for asynchronous frame writing
- add_frame(frame): Non-blocking frame addition to queue (drops frames if queue full to prevent backpressure).
- _write_frames(): Worker thread that continuously writes frames from queue to video file.
- stop_recording(): Graceful shutdown of recording thread and resource cleanup.

Purpose: Enables on-demand recording of monitoring sessions without impacting real-time performance.

5.5.8. MainTrackCurrent.py - Main Application Orchestrator

The central controller that integrates all components and manages the real-time processing loop.

Core Processing:

1. Initialization Phase:
   - Parses command-line arguments
   - Resolves model archive path via ensure_nn_archive()
   - Initializes monitoring, tracking, database, and recording systems
   - Configures DepthAI pipeline with all required nodes
2. Frame Processing Loop:

   while True:

   monitor.start_frame() # Start timin

   # 1. Frame Acquisition

   msg = parser_output_queue.get() # NN detections

   color_msg = colorFrame.get()   # RGB frame

   # 2. Object Tracking

   tracked_detections = tracker.get_tracked_results(msg.detections, color_image.shape)

   # 3. Spatial Configuration

   if tracked_detections:

   for each fish:

   extract keypoints (head, tail)

generate intermediate points

configure spatial ROIs for each point

4. 3D Spatial Calculation

send spatial config to DepthAI

receive 3D coordinates for all points

5. Biometric Analysis

for each fish:

collect all 3D points

fit polynomial curve → calculate length

compute centroid → track movement

determine activity status

6. Data Logging

if update_interval_reached:

DB.save_data_to_db(fish_id, length, distance, status, etc.)

7. Visualization

annotate frame with boxes, IDs, metrics

display system start

8. Recording

if recording_active:

recorder.add_frame(annotated_frame)

monitor.end_frame() # End timing

3. Clean up phase:
   - Close database connection
   - Stop monitoring
   - Release pipeline resources
   - Stop recording if active

Key Features:

- Time-Based Status Classification: Fish are classified as active/lethargic/dead based on configurable time thresholds of no movement.
- Frame-Based Timeout: Fish not seen for FRAME_TIMEOUT frames are marked inactive in database.
- Periodic Database Updates: Updates occur every N frames to balance data freshness with performance.
- Dual Visualization: Supports both local OpenCV display and remote web visualizer.
- Hotkey Controls: 'r' toggles recording, 'x' exits application.

5.5.9. System Configuration and Parameters

The system is configurable through several key parameters:

Tracking Parameters: track_thresh, track_buffer, minimum_matching_threshold.

Behavior Classification: MINIMAL_MOTION_THRESHOLD, LETHARGY_TIMEOUT_MINUTES, DEAD_TIMEOUT_MINUTES.

Performance: fps_limit, FRAME_TIMEOUT, history_size for monitoring.

Model: confidence_threshold, iou_threshold for detection filtering.

Biometrics: num_points for intermediate point generation, polynomial degree for curve fitting.

6. Result

Owing to budgetary and equipment limitations, my evaluation relied on a modest sample of two snakehead fish. This restricted scope, while not ideal for statistical validation, allowed me to assess whether the core pipeline functioned as intended. My collaborator, Mr. Nguyen Huu Minh Quang, and I conducted these tests together. Notably, because the OAK-D camera is not waterproof, all trials were necessarily performed in air, with the fish temporarily placed in a shallow container. These conditions mean the system's performance in its intended underwater setting remains untested; the results presented here should therefore be interpreted as a preliminary proof of concept rather than a definitive performance benchmark.

First, we had to measure the real length of the fish for compare to length from the system.

Figure 12. fish 1

Figure 13. Fish 2

Fish 1: real length is 28 cm.

Fish 2: real length is 26 cm.

For the detection and length estimation:

Given that the accuracy of the length measurement appears to hinge on two factors—precise keypoint placement and reliable depth estimation—the results have been categorized into two distinct scenarios. The first represents optimal conditions, where the model correctly identifies the head and tail. The second covers suboptimal detection, such as when keypoints are misplaced or the model struggles under poor lighting. This split helps isolate how error propagates from detection into the final metric. Furthermore, to evaluate the practical benefit of the more complex approach, both length estimation methods—the polynomial curve fitting and the simple Euclidean distance between points—are compared under each scenario.

Best condition		
Fish	Calculated Curve length	Calculated Euclidean length
Fish 1	27.5 cm	27.21 cm
Fish 1	27.25 cm	26.98 cm
Fish 1	28.95 cm	27.73 cm
Fish 1	27.96 cm	27.7 cm

Best condition		
Fish	Calculated Curve length	Calculated Euclidean length
Fish 2	25.96 cm	25.61 cm
Fish 2	26.26 cm	25.74 cm
Fish 2	25.49 cm	25.35 cm
Fish 2	25.58 cm	24.95 cm

In the best condition, the overall length accuracy get from the two fish is more than 94% with margin error of $\pm$1-4% on fish 1 and $\pm$1-2% on fish 2.

For the worst condition, include when the model can't work well and low light condition:

worst condition		
Fish	Calculated Curve length	Calculated Euclidean length
Fish 1	26.5 cm	26.15 cm
Fish 1	26.28 cm	25.98 cm
Fish 1	30.7 cm	27.73 cm
Fish 1	25.08 cm	25.01 cm

Worst condition		
Fish	Calculated Curve length	Calculated Euclidean length
Fish 2	24.86 cm	24.31 cm
Fish 2	23.45 cm	23.34 cm
Fish 2	25.08 cm	24.95 cm
Fish 2	23.58 cm	23.20 cm

From the table in the worst condition we tested, accuracy reduce to around 92% or sometime 85% with margin error ±4% - 10% for fish 1 and ±5% - 11 % for fish 2.

In comparison, the curve fitting one is more accurate than Euclidean length about 1% in consideration of the curve of the fish at that time. However, the curve fitting has higher upper margin error than the Euclidean length because the curve fitting method tend to overestimate the fish length.

For the overall system (host and OAK-D camera) performance:

- The average FPS: 9 FPS
- The average frame time: 111 ms
- The average CPU usage: 52%

The FPS can be heavily affected by the number of fish detected.

Figures 14. Demo for status assignment system

7. Discussion and Future work

The system presented here demonstrates that real-time, stereo vision-based fish detection and biometric measurement is feasible on edge hardware, even with the constraints of a low-power host and a compact camera module. In optimal conditions—good lighting, clear water, and reliable keypoint placement—length estimates consistently fell within 5–6% of manual measurements, which for many aquaculture monitoring purposes could be considered acceptable. The curve-fitting method generally provided a slight improvement over simple Euclidean distance, particularly when fish exhibited natural body curvature, though its tendency to occasionally overestimate length suggests the algorithm might be sensitive to noise in the depth data or to the specific polynomial degree chosen.

That said, the performance I observed under suboptimal conditions reveals several tangible limitations. The accuracy decrease in low light or turbid water was expected, but the extent to which keypoint detection degraded was notable—sometimes confusing the head and tail, or failing to place points on the fish body entirely. This isn't entirely surprising; the YOLOv8n-pose model was trained on a limited dataset, and pose estimation for slender, fast-moving targets remains a challenging computer vision problem. The system's reliance on only two keypoints per fish is both a strength (reducing computational load) and a weakness: when one point is misplaced, the subsequent 3D projection and length calculation inherit that error. In practice, this meant that in "worst-case" test scenarios, errors could exceed 10%, which for precise biomass estimation might be too high.

Performance-wise, the pipeline managed around 9 FPS on a dual-core laptop, which is sufficient for near-real-time monitoring but leaves little headroom for scaling to denser populations or adding more complex analytics. The bottleneck appeared to be less the OAK-D's VPU—which handled inference and depth well—and more the host-side tasks: coordinate projection, curve fitting, tracking, and database operations. This suggests further

optimizing the host code (perhaps with just-in-time compilation or parallel processing), could improve throughput.

The behavioral status classification, while functional, is admittedly rudimentary. Using a time-based threshold instead of a frame-based one avoids coupling behavior to FPS, but the underlying heuristic—measuring displacement of a computed centroid—does not account for common behaviors like fin fluttering or gill movement. In our tests, a fish holding stationery in a current would sometimes be marked "lethargic," while subtle vibrations from equipment could reset the timer. This isn't a failure of the implementation so much as a reflection of the simplicity of the chosen metric; in a production setting, such status outputs would be better used as triggers for human review rather than as definitive health indicators.

Looking forward, several directions seem promising for improving the system. First and most impactful would be to train a more robust pose estimation model. The current model works, but expanding the training dataset to include more varied poses, lighting conditions, water clarities, and fish species would likely improve keypoint stability. Incorporating more than two keypoints—perhaps adding a mid-body point—could also provide better curvature estimation and allow for more graceful degradation when one point is occluded.

Second, the system's performance on the edge could be enhanced. Porting the entire pipeline to a dedicated edge device like a Raspberry Pi 4 or Jetson Nano would better realize the vision of a low-power, stand-alone monitoring unit. This would likely require further model quantization, perhaps using INT8 precision, and rewriting some of the host-side processing in C++ or using hardware acceleration.

Third, the curve-fitting algorithm, while mathematically sound, is computationally expensive for a real-time context. Exploring simpler or more numerically stable approximations—like spline fitting or adaptive polyline segmentation—might reduce CPU load without significantly compromising accuracy. Additionally, integrating a confidence score for each length estimate, based on keypoint confidence and depth consistency, would help users gauge measurement reliability.

Finally, the testing environment here was limited—two fish, in air, with a non-waterproof camera. While this served as a valid proof of concept, the system's true utility will only be clear after long-term deployment in actual aquaculture settings: submerged in water, facing biofouling, diurnal light changes, and much higher fish densities. Such deployments would likely uncover new failure modes, from occlusion in dense schools to reflection artifacts on the water surface, providing the necessary data to iteratively refine both the model and the processing pipeline.

8. Conclusion

In conclusion, this project suggests that affordable, edge-based stereo vision can indeed automate fish biometrics and basic behavior tracking, but it also highlights the gaps between a working prototype and a robust field-ready system. The code works, the measurements are plausible, and the architecture is modular enough to extend—but each component, from the AI model to the curve-fitting logic, has room for improvement. The next steps would involve not just algorithmic tweaks, but also real-world validation and a commitment to iterative, problem-driven refinement.

9. References

1. Yaozhen Yu, Hao Zhang, Fei Yuan “Key point detection method for fish size measurement based on deep learning”. 2023
2. Petter Risholm, Ahmed Mohammed, Trine Kirkhus, Sigmund Clausen, Leonid Vasilyev, Ole Folkedal, Øistein Johnsen, Karl Henrik Haugholt, Jens Thielemann, “Automatic length estimation of free-swimming fish using an underwater 3D range-gated camera, Aquacultural Engineering”, 2022
3. Pinhole Distance - A Python Library for Camera Distance Calculation. https://shorturl.at/DtGWB
4. NGO DUC THINH – Vietnamese German University student - Matriculation number: 1357053 “Real-Time Underwater Monitoring Implementing a System for Fish Detection, Counting, and Cloud Data Integration using OAK-D AI Camera”. 2024
5. Oak D stereo camera document Hub– Luxonis.
6. Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). “Edge Computing: Vision and Challenges. IEEE Internet of Things Journal”
7. Nguyen Huu Minh Quang’s github. https://github.com/QuangHuu/Fishbt1/tree/main

10. Appendix

My github for the code and demo video: https://github.com/Tea-Catj/FishID-ML-on-edge-device-.git